

What Happens When You Memoize Impure Functions?

You should not add the `@lru_cache` to functions that are not pure, meaning they are nondeterministic or have side effects. Memoization saves time by skipping the code in the function and returning the previously cached return value. This is fine for pure functions but can cause various bugs for impure functions.

In nondeterministic functions, such as a function that returns the current time, memoization causes the function to return incorrect results. For functions with side effects, such as printing text to the screen, memoization causes the function to skip the intended side effect. The *doNotMemoize.py* program demonstrates what happens when the `@lru_cache` function decorator (described in the previous section) memoizes these impure functions:

Python

```
import functools, time, datetime

@functools.lru_cache()
def getCurrentTime():
    # This nondeterministic function returns different values each time
    # it's called.
    return datetime.datetime.now()

@functools.lru_cache()
def printMessage():
    # This function displays a message on the screen as a side effect.
    print('Hello, world!')

print('Getting the current time twice:')
print(getCurrentTime())
print('Waiting two seconds...')
time.sleep(2)
print(getCurrentTime())

print()

print('Displaying a message twice:')
```